

L I N F O 2 3 4 7

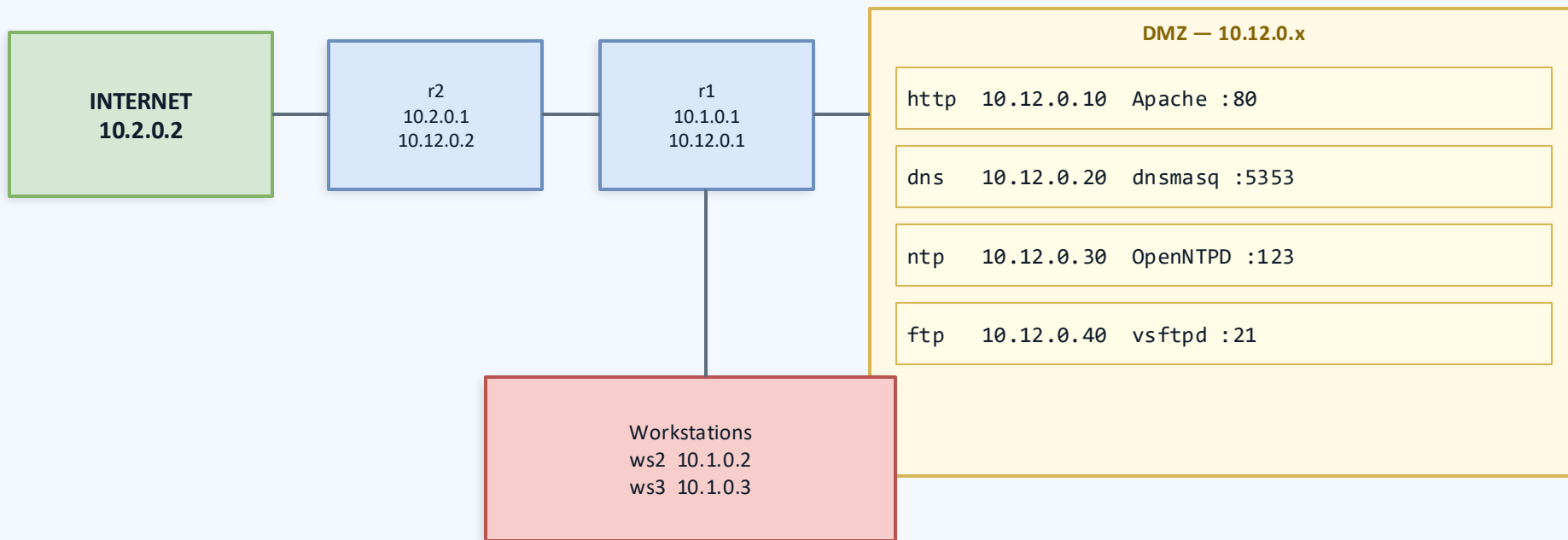
Network Attacks

& Protections

Mininet Enterprise Network Topology | nftables Firewall Protection

Network Scan · FTP Brute Force · ARP Poisoning · Reflected DDoS · Botnet

Network Topology & Basic Enterprise Protection



Basic Enterprise Firewall Policies

Zone	Policy	Enforced on
Workstations	Can ping & initiate connections to any host	
DMZ Servers	output policy drop only established/related replies allowed	http, dns, ntp, ftp
Internet	Cannot ping or connect to workstations (10.1.0.0/24)	r2 forward chain

✂ Attack

Attacker probes both subnets from the internet to discover live hosts and open services.

- Sends ICMP echo requests to every IP to discover live hosts SYN-ACK means open
- Sends TCP SYN to ports 21, 22, 80, 5353 on each host
- SYN-ACK = OPEN | RST = CLOSED | No reply = FILTERED

Attacker gets a full map: which hosts exist and which services are exposed

🛡 Protection - nftables

Reduce scan visibility at the perimeter router r2

- Rule 1: `iifname r2-eth0 icmp echo-request limit rate over 3/sec drop`
- Rule 2: `iifname r2-eth0 tcp flags syn tcp dport != {80,21} drop`
- Only HTTP (80) and FTP (21) remain visible from internet

SSH and DNS completely hidden attack surface drastically reduced

Attack 1 — Network Scan

internet host → scans 10.12.0.0/24 & 10.1.0.0/24

```
mininet@mininet-vm: ~  
/usr/lib/python3/dist-packages/scapy/layers/ipsec.py:469: CryptographyDeprecationWarning: Blowfish has been deprecated and will be removed in a future release  
cipher=algorithms.Blowfish,  
/usr/lib/python3/dist-packages/scapy/layers/ipsec.py:483: CryptographyDeprecationWarning: CAST5 has been deprecated and will be removed in a future release  
cipher=algorithms.CAST5,  
[*] ICMP scan on 10.12.0.0/24  
[+] 10.12.0.1 is UP (ICMP)  
[+] 10.12.0.2 is UP (ICMP)  
[+] 10.12.0.10 is UP (ICMP)  
[+] 10.12.0.20 is UP (ICMP)  
[+] 10.12.0.30 is UP (ICMP)  
[+] 10.12.0.40 is UP (ICMP)  
[*] TCP SYN scan on 10.12.0.0/24 ports [21, 22, 80, 5353]  
[-] 10.12.0.1:21 is CLOSED  
[-] 10.12.0.1:22 is CLOSED  
[-] 10.12.0.1:80 is CLOSED  
[-] 10.12.0.1:5353 is CLOSED  
[-] 10.12.0.2:21 is CLOSED  
[-] 10.12.0.2:22 is CLOSED  
[-] 10.12.0.2:80 is CLOSED  
[-] 10.12.0.2:5353 is CLOSED  
[-] 10.12.0.10:21 is CLOSED  
[+] 10.12.0.10:22 is OPEN — x(SH) exposed  
[-] 10.12.0.10:80 is CLOSED  
[-] 10.12.0.10:5353 is CLOSED  
[-] 10.12.0.20:21 is CLOSED  
[-] 10.12.0.20:22 is CLOSED  
[-] 10.12.0.20:80 is CLOSED  
[+] 10.12.0.20:5353 is OPEN — (DNS) exposed  
[-] 10.12.0.30:21 is CLOSED  
[+] 10.12.0.30:22 is OPEN  
[-] 10.12.0.30:80 is CLOSED  
[-] 10.12.0.30:5353 is CLOSED  
[+] 10.12.0.40:21 is OPEN — (SSH) exposed  
[+] 10.12.0.40:22 is OPEN  
[-] 10.12.0.40:80 is CLOSED  
[-] 10.12.0.40:5353 is CLOSED
```

[+] 10.12.0.10:21 is CLOSED

mininet@mininet-vm: ~

* Management: <https://landscape.canonical.com>
* Support: <https://ubuntu.com/advantage>

* Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support on 31 May 2025.

For more details see:
<https://ubuntu.com/20-04>
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue May 5 12:10:33 2026 from 192.168.56.1

mininet@mininet-vm:~\$ sudo mnexec -a \$(pgrep -f "mininet:internet") python3 ~/LINF02347/attacks/network_scan.py

[*] ICMP scan on 10.12.0.0/24

[+] 10.12.0.1 is UP (ICMP)
[+] 10.12.0.2 is UP (ICMP)
[+] 10.12.0.10 is UP (ICMP)
[+] 10.12.0.20 is UP (ICMP)
[+] 10.12.0.30 is UP (ICMP)
[+] 10.12.0.40 is UP (ICMP)
[+] 10.12.0.195 is UP (ICMP)

[*] TCP SYN scan on 10.12.0.0/24 ports [21, 22, 80, 5353]

[-] 10.12.0.1:21 is CLOSED
[-] 10.12.0.1:80 is CLOSED
[-] 10.12.0.2:21 is CLOSED
[-] 10.12.0.2:22 is CLOSED
[-] 10.12.0.2:80 is CLOSED
[-] 10.12.0.2:5353 is CLOSED
[-] 10.12.0.10:21 is CLOSED
[+] 10.12.0.10:80 is OPEN — ✓ HTTP
[-] 10.12.0.20:21 is CLOSED
[-] 10.12.0.20:80 is CLOSED
[-] 10.12.0.30:21 is CLOSED
[-] 10.12.0.30:80 is CLOSED
[+] 10.12.0.40:21 is OPEN — ✓ FTP
[-] 10.12.0.40:80 is CLOSED

only
rest hidden

✂ Attack

Attacker tries all combinations from a username/password wordlist against ftp:21

- Each attempt = 1 new TCP connection: SYN → USER → PASS → check response code
- Response 230 = Login successful | 530 = Login failed
- Without protection: finds mininet:mininet credentials in ~17 seconds

Real wordlists have 100,000+ passwords, success inevitable without protection

🛡 Protection — nftables

Slow the attack to impractical speed using an nftables meter on the FTP server.

- Rule: tcp dport 21 ct state new meter ftp_limit { ip saddr limit rate 3/minute burst 3 } accept
- Fallthrough: tcp dport 21 ct state new drop
- Max 3 new connections/minute per source IP, any IP not just attacker
- 35 passwords → 11 min | 100k passwords → 23 days to exhaust

attacker must wait 1 minute between every 3 guesses — wordlist becomes useless.

Attack 2 — FTP Brute Force

internet host → ftp server 10.12.0.40:21

```
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:internet") python3 ~/LINF02347/attacks/ftp_bruteforce.py
[*] Starting FTP brute force on 10.12.0.40:21
[*] Trying admin:123456
[*] Trying admin:password
[*] Trying admin:admin
[*] Trying admin:root
[*] Trying admin:mininet
[*] Trying admin:ftp
[*] Trying admin:toor
[*] Trying root:123456
[*] Trying root:password
[*] Trying root:admin
[*] Trying root:root
[*] Trying root:mininet
[*] Trying root:ftp
[*] Trying root:toor
[*] Trying ftp:123456
[*] Trying ftp:password
[*] Trying ftp:admin
[*] Trying ftp:root
[*] Trying ftp:mininet
[*] Trying ftp:ftp
[*] Trying ftp:toor
[*] Trying user:123456
[*] Trying user:password
[*] Trying user:admin
[*] Trying user:root
[*] Trying user:mininet
[*] Trying user:ftp
[*] Trying user:toor
[*] Trying mininet:123456
[*] Trying mininet:password
[*] Trying mininet:admin
[*] Trying mininet:root
[*] Trying mininet:mininet
[+] SUCCESS! mininet:mininet
mininet@mininet-vm:~$
```

```
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:internet") python3 ~/LINF02347/attacks/ftp_bruteforce.py
[*] Trying mininet:mininet
[*] SUCCESS! mininet:mininet
mininet@mininet-vm:~$ sudo bash ~/LINF02347/protectations/protect_ftp_bruteforce.sh
Applying FTP brute force protection on ftp server...
FTP brute force protection applied!
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:internet") python3 ~/LINF02347/attacks/ftp_bruteforce.py
[*] Starting FTP brute force on 10.12.0.40:21
[*] Trying admin:123456
[*] Trying admin:password
[*] Trying admin:admin
[*] Trying admin:root
[*] Trying admin:mininet
[*] Trying admin:ftp
[*] Trying admin:toor
[*] Trying root:123456
[*] Trying root:password
[*] Trying root:admin
[*] Trying root:root
[*] Trying root:mininet
[*] Trying root:ftp
[*] Trying root:toor
[*] Trying ftp:123456
[*] Trying ftp:password
[*] Trying ftp:admin
[*] Trying ftp:root
[*] Trying ftp:mininet
[*] Trying ftp:ftp
[*] Trying ftp:toor
[*] Trying user:123456
[*] Trying user:password
[*] Trying user:admin
[*] Trying user:root
[*] Trying user:mininet
[*] Trying user:ftp
[*] Trying user:toor
[*] Trying mininet:123456
[*] Trying mininet:password
[*] Trying mininet:admin
[*] Trying mininet:root
[*] Trying mininet:mininet
[*] Trying mininet:ftp
[*] Trying mininet:toor
[*] No valid credentials found after 35 attempts
mininet@mininet-vm:~$
```

Attack 3 ARP Cache Poisoning

ws3 → poisons ws2's ARP cache (Man in the Middle)

✂ Attack

ws3 poisons ws2's ARP cache to redirect traffic meant for the gateway.

- ARP is stateless - any host can send ARP replies without being asked
- ws3 sends fake ARP reply to ws2: 'I am 10.1.0.1 (gateway), my MAC is ws3_MAC'
- ws2 stores the fake entry - all traffic now flows to ws3 instead of r1
- ws3 becomes Man in the Middle: intercept, read or modify ws2's traffic silently

Before attack: 10.1.0.1 → correct MAC

After attack: 10.1.0.1 → ws3 MAC (poisoned)

🛡 Protection - nftables

MAC pinning at the ARP layer on both workstations.

- Applied on ws2 and ws3 - nftables ARP table filtering
- Rule: arp operation reply arp saddr ip 10.1.0.1 arp saddr ether != REAL_GW_MAC drop
- Drops ARP replies claiming to be the gateway with a wrong MAC address
- Legitimate ARP from real gateway still accepted normally

After protection: fake ARP replies rejected - ws2 ARP cache stays clean

Attack 3 ARP Cache Poisoning

ws3 → poisons ws2's ARP cache (Man in the Middle)

```
Last login: Tue May 5 12:37:32 2026 from 192.168.56.1
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws3") python3 ~/LINFO2347/attacks/arp_poison.py
[*] Getting real MAC of 10.1.0.1...
[*] Real MAC of http server: ae:df:24:09:d4:7a

[*] Starting ARP poisoning...
[*] Victim: 10.1.0.2 (ee:fe:87:27:f2:06)
[*] Impersonating: 10.1.0.1
```

ATTACK

```
[*] Sending 10 poison packets...
[+] Sent poison packet 1/10
[+] Sent poison packet 2/10
[+] Sent poison packet 3/10
[+] Sent poison packet 4/10
[+] Sent poison packet 5/10
[+] Sent poison packet 6/10
[+] Sent poison packet 7/10
[+] Sent poison packet 8/10
[+] Sent poison packet 9/10
[+] Sent poison packet 10/10
```

```
[*] Restoring ARP tables...
[+] Done! ARP tables restored.
mininet@mininet-vm:~$
```

```
] Trying user:123456
] Trying user:password
] Trying user:admin
] Trying user:root
] Trying user:mininet
] Trying user:ftp
] Trying user:toor
] Trying mininet:123456
] Trying mininet:password
] Trying mininet:admin
] Trying mininet:root
] Trying mininet:mininet
] Trying mininet:ftp
] Trying mininet:toor
```

] No valid credentials found after 35 attempts

```
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws2") arp -n
ldress      Hwtype  Hwaddress      Flags Mask      Iface
1.1.0.1     ether   ae:df:24:09:d4:7a  C              ws2-eth0
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws2") arp -n
ldress      Hwtype  Hwaddress      Flags Mask      Iface
1.1.0.1     ether   d2:0f:ec:f5:e3:5d  C              ws2-eth0
1.1.0.3     ether   d2:0f:ec:f5:e3:5d  C              ws2-eth0
mininet@mininet-vm:~$
```

before
R1
ws3
(Same MAC)

```
*) Getting real MAC of 10.1.0.1...
*) Real MAC of http server: ae:df:24:09:d4:7a
```

```
*) Starting ARP poisoning...
*) Victim: 10.1.0.2 (ee:fe:87:27:f2:06)
*) Impersonating: 10.1.0.1
```

AFTER
protection

```
*) Sending 10 poison packets...
[+] Sent poison packet 1/10
[+] Sent poison packet 2/10
[+] Sent poison packet 3/10
[+] Sent poison packet 4/10
[+] Sent poison packet 5/10
[+] Sent poison packet 6/10
[+] Sent poison packet 7/10
[+] Sent poison packet 8/10
[+] Sent poison packet 9/10
[+] Sent poison packet 10/10
```

```
*) Restoring ARP tables...
+) Done! ARP tables restored.
mininet@mininet-vm:~$
```

```
*) Trying mininet:ftp
*) Trying mininet:toor
```

*) No valid credentials found after 35 attempts

```
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws2") arp -n
ldress      Hwtype  Hwaddress      Flags Mask      Iface
0.1.0.1     ether   ae:df:24:09:d4:7a  C              ws2-eth0
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws2") arp -n
ldress      Hwtype  Hwaddress      Flags Mask      Iface
0.1.0.1     ether   d2:0f:ec:f5:e3:5d  C              ws2-eth0
0.1.0.3     ether   d2:0f:ec:f5:e3:5d  C              ws2-eth0
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws2") arp -n
ldress      Hwtype  Hwaddress      Flags Mask      Iface
0.1.0.3     ether   d2:0f:ec:f5:e3:5d  C              ws2-eth0
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ws2") arp -n
ldress      Hwtype  Hwaddress      Flags Mask      Iface
0.1.0.3     ether   d2:0f:ec:f5:e3:5d  C              ws2-eth0
mininet@mininet-vm:~$
```

Correct MAC

Attack 4 -Reflected DDoS -UDP Amplification

internet → spoofs http IP → ntp reflector floods
http

✂ Attack

Attacker spoofs source IP as victim (10.12.0.10) and sends 3-byte GET to reflector

- Reflector (ntp host) thinks victim sent request → replies with 1000 bytes to victim
- Victim flooded with traffic it never requested - attacker stays hidden
- Amplification: 200×3 bytes sent = 600B → 200×1000 bytes received = 200KB (333× amplification)

Note: dnsmasq (port 5353) and OpenNTPD both did not respond to external queries in mininet. We deployed a custom UDP reflector on ntp to fully demonstrate the spoofing + amplification mechanism — identical to real DNS/NTP amplification attacks

🛡 Protection - nftables

Ingress filtering on the victim (http server) to drop amplified replies before they reach the application.

- Adds an input rule: drop all UDP packets with sport 12345
- Rule: ip protocol udp udp sport 12345 counter drop
- Drops ALL unsolicited UDP from reflector port - victim never initiated this traffic
- Counter provides hard proof: packets 200 bytes 205600 drop
- Normal HTTP (port 80) traffic completely unaffected

Amplified UDP packets are discarded at the NIC; HTTP service stays responsive.

Attack 4 — Reflected DDoS — UDP Amplification

internet → spoofs http IP → ntp reflector floods
http

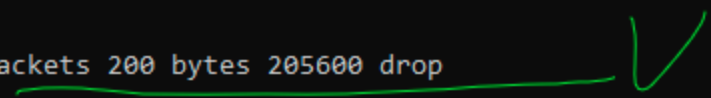
```
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
mininet@mininet-vm:~$ nano ~/LINF02347/attacks/udp_reflector.py
mininet@mininet-vm:~$ nano ~/LINF02347/attacks/reflected_ddos.py
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:ntp") python3 ~/LINF02347/attacks/udp_reflector.py
[*] UDP Reflector listening on port 12345
[*] Will send large replies to any request
[*] Got request from ('10.12.0.10', 999) - sending amplified reply
[*] Got request from ('10.12.0.10', 36243) - sending amplified reply
[*] Got request from ('10.12.0.10', 10767) - sending amplified reply
[*] Got request from ('10.12.0.10', 58840) - sending amplified reply
[*] Got request from ('10.12.0.10', 6416) - sending amplified reply
[*] Got request from ('10.12.0.10', 45485) - sending amplified reply
[*] Got request from ('10.12.0.10', 19758) - sending amplified reply
[*] Got request from ('10.12.0.10', 52138) - sending amplified reply
[*] Got request from ('10.12.0.10', 56463) - sending amplified reply
[*] Got request from ('10.12.0.10', 34110) - sending amplified reply
[*] Got request from ('10.12.0.10', 4624) - sending amplified reply
[*] Got request from ('10.12.0.10', 22502) - sending amplified reply
[*] Got request from ('10.12.0.10', 264) - sending amplified reply
[*] Got request from ('10.12.0.10', 46200) - sending amplified reply
[*] Got request from ('10.12.0.10', 15001) - sending amplified reply
[*] Got request from ('10.12.0.10', 24976) - sending amplified reply
[*] Got request from ('10.12.0.10', 5315) - sending amplified reply
[*] Got request from ('10.12.0.10', 44687) - sending amplified reply
[*] Got request from ('10.12.0.10', 44306) - sending amplified reply
[*] Got request from ('10.12.0.10', 49944) - sending amplified reply
[*] Got request from ('10.12.0.10', 26555) - sending amplified reply
[*] Got request from ('10.12.0.10', 59132) - sending amplified reply
[*] Got request from ('10.12.0.10', 11081) - sending amplified reply
[*] Got request from ('10.12.0.10', 3170) - sending amplified reply
[*] Got request from ('10.12.0.10', 6895) - sending amplified reply
[*] Got request from ('10.12.0.10', 45968) - sending amplified reply
[*] Got request from ('10.12.0.10', 10275) - sending amplified reply
[*] Got request from ('10.12.0.10', 23385) - sending amplified reply
[*] Got request from ('10.12.0.10', 37288) - sending amplified reply
[*] Got request from ('10.12.0.10', 12627) - sending amplified reply
[*] Got request from ('10.12.0.10', 16774) - sending amplified reply
[*] Got request from ('10.12.0.10', 3919) - sending amplified reply
[*] Got request from ('10.12.0.10', 43172) - sending amplified reply
[*] Got request from ('10.12.0.10', 27331) - sending amplified reply
[+] Sent 20/100 spoofed NTP requests
[+] Sent 30/100 spoofed NTP requests
[+] Sent 40/100 spoofed NTP requests
[+] Sent 50/100 spoofed NTP requests
[+] Sent 60/100 spoofed NTP requests
[+] Sent 70/100 spoofed NTP requests
[+] Sent 80/100 spoofed NTP requests
[+] Sent 90/100 spoofed NTP requests
[+] Attack complete!
[+] NTP server sent large replies to victim 10.12.0.10
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:http") tcpdump -i http-eth0 udp port 12345 -c 20 -v
tcpdump: listening on http-eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
17:21:46.597544 IP (tos 0x0, ttl 63, id 1, offset 0, flags [none], proto UDP (17), length 31)
    10.12.0.10.999 > 10.12.0.30.12345: UDP, length 3
17:21:46.599904 IP (tos 0x0, ttl 64, id 19697, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.999: UDP, length 1000
17:21:46.631938 IP (tos 0x0, ttl 64, id 19703, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.36243: UDP, length 1000
17:21:46.675250 IP (tos 0x0, ttl 64, id 19709, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.10767: UDP, length 1000
17:21:46.708596 IP (tos 0x0, ttl 64, id 19715, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.58840: UDP, length 1000
17:21:46.741524 IP (tos 0x0, ttl 64, id 19717, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.6416: UDP, length 1000
17:21:46.783853 IP (tos 0x0, ttl 64, id 19724, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.45485: UDP, length 1000
17:21:46.812616 IP (tos 0x0, ttl 64, id 19727, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.19758: UDP, length 1000
17:21:46.852379 IP (tos 0x0, ttl 64, id 19733, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.52138: UDP, length 1000
17:21:46.888878 IP (tos 0x0, ttl 64, id 19742, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.56463: UDP, length 1000
17:21:46.925735 IP (tos 0x0, ttl 64, id 19749, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.34110: UDP, length 1000
17:21:46.954534 IP (tos 0x0, ttl 64, id 19750, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.4624: UDP, length 1000
17:21:46.993028 IP (tos 0x0, ttl 64, id 19751, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.22502: UDP, length 1000
17:21:47.025295 IP (tos 0x0, ttl 64, id 19753, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.264: UDP, length 1000
17:21:47.068139 IP (tos 0x0, ttl 64, id 19762, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.46200: UDP, length 1000
17:21:47.119405 IP (tos 0x0, ttl 64, id 19773, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.15001: UDP, length 1000
17:21:47.169211 IP (tos 0x0, ttl 64, id 19776, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.24976: UDP, length 1000
17:21:47.202380 IP (tos 0x0, ttl 64, id 19779, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.5315: UDP, length 1000
17:21:47.245023 IP (tos 0x0, ttl 64, id 19783, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.44687: UDP, length 1000
17:21:47.278620 IP (tos 0x0, ttl 64, id 19789, offset 0, flags [DF], proto UDP (17), length 1028)
    10.12.0.30.12345 > 10.12.0.10.44306: UDP, length 1000
20 packets captured
0 packets received by filter
0 packets dropped by kernel
```

Attack 4 Reflected DDoS UDP Amplification

internet → spoofs http IP → ntp reflector floods
http

```
mininet@mininet-vm:~$ sudo mnexec -a $(pgrep -f "mininet:http") nft list ruleset
table inet filter {
    chain output {
        type filter hook output priority filter; policy drop;
        ct state established,related accept
        oif "lo" accept
        icmp type echo-reply accept
    }

    chain input {
        type filter hook input priority filter; policy accept;
        iif "lo" accept
        ct state established,related accept
        ip protocol udp udp sport 12345 counter packets 200 bytes 205600 drop
    }
}
mininet@mininet-vm:~$
```



Attack 5 Botnet HTTP Flood (C&C)

internet C&C → commands ws2 & ws3 bots →
floods http

✂ Attack

C&C server runs on internet host, waits for bots to connect then issues flood command towards the HTTP server

- ws2 and ws3 act as compromised bots they connect to C&C and await orders
- C&C launches after 10 seconds: bots repeatedly send HTTP requests to 10.12.0.10:80
- Bots (ws2, ws3) repeatedly open TCP connections to http:80 and send HTTP GET requests

traffic comes from multiple legitimate-looking IPs which is hard to distinguish from normal traffic since each bot individually looks like a real user

🛡 Protection — nftables

Per-source-IP rate limiting on every DMZ server using nftables meters.

- Applied on DMZ servers (http, ftp, dns, ntp) nftables meter per source IP
- RULE: `nft add rule inet filter input tcp dport 80 ct state new meter flood size 65535 { ip saddr limit rate over 10/second } reject`
- Rejects excessive new connections per source IP on all service ports
- Rate limit: any source IP sending too many new connections per second is throttled
- Allows at most 10 new connections per second per source IP

Bots see more rejected/failed connections HTTP server stays responsive

Attack 5 Botnet HTTP Flood (C&C)

internet C&C → commands ws2 & ws3 bots → floods http

```

mininet: interne python3 ~/LINFO2347/attacks/botnet_cnc.py &
mininet: ws2 python3 ~/LINFO2347/attacks/bot.py &
mininet: ws3 python3 ~/LINFO2347/attacks/bot.py &
mininet: ws2 jobs

[BOT-1] Starting up...
[BOT-1] Connecting to C&C at 10.2.0.2:9999
[BOT-1] [OK] Connected to C&C! Waiting for commands...
[BOT-1] Received command: ATTACK 10.12.0.10 80
[BOT-1] Launching attack on 10.12.0.10:80
[BOT-1] Starting HTTP flood on 10.12.0.10:80
[BOT-1] [OK] 10 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 20 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 30 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 40 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 50 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 60 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 70 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 80 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 90 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 100 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 110 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 120 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 130 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 140 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 150 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 160 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 170 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 180 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 190 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 200 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 210 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 220 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 230 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 240 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 250 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 260 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 270 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 280 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 290 requests sent | [FAIL] 0 failed
[1]+  Running                  python3 ~/LINFO2347/attacks/bot.py &
mininet: ws3 jobs

[BOT-1] Starting up...
[BOT-1] Connecting to C&C at 10.2.0.2:9999
[BOT-1] [OK] Connected to C&C! Waiting for commands...
[BOT-1] Received command: ATTACK 10.12.0.10 80
[BOT-1] Launching attack on 10.12.0.10:80
[BOT-1] Starting HTTP flood on 10.12.0.10:80
[BOT-1] [OK] 10 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 20 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 30 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 40 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 50 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 60 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 70 requests sent | [FAIL] 0 failed
[BOT-1] [OK] 80 requests sent | [FAIL] 0 failed

```

```
mininet> interne python3 ~/LINF02347/attacks/botnet_cnc.py &
mininet> ws2 python3 ~/LINF02347/attacks/bot.py &
mininet> ws3 python3 ~/LINF02347/attacks/bot.py &
mininet> ws2 jobs
[BOT-1] Starting up...
[BOT-1] Connecting to C&C at 10.2.0.2:9999
[BOT-1] [OK] Connected to C&C! Waiting for commands...
[BOT-1] Received command: ATTACK 10.12.0.10 80
[BOT-1] Launching attack on 10.12.0.10:80
[BOT-1] Starting HTTP flood on 10.12.0.10:80
[BOT-1] [OK] 10 requests sent | [FAIL] 6 failed
[BOT-1] [OK] 20 requests sent | [FAIL] 8 failed
[1]+  Running                  python3 ~/LINF02347/attacks/bot.py &
mininet> ws3 jobs
[BOT-1] Starting up...
[BOT-1] Connecting to C&C at 10.2.0.2:9999
[BOT-1] [OK] Connected to C&C! Waiting for commands...
[BOT-1] Received command: ATTACK 10.12.0.10 80
[BOT-1] Launching attack on 10.12.0.10:80
[BOT-1] Starting HTTP flood on 10.12.0.10:80
[BOT-1] [OK] 10 requests sent | [FAIL] 6 failed
[BOT-1] [OK] 20 requests sent | [FAIL] 8 failed
[BOT-1] [FAIL] 10 requests failed (server overwhelmed!)
[BOT-1] [OK] 30 requests sent | [FAIL] 10 failed
[BOT-1] [OK] 40 requests sent | [FAIL] 12 failed
[BOT-1] [OK] 50 requests sent | [FAIL] 14 failed
[1]+  Running                  python3 ~/LINF02347/attacks/bot.py &
mininet> ws2 jobs
[BOT-1] [FAIL] 10 requests failed (server overwhelmed!)
[BOT-1] [OK] 30 requests sent | [FAIL] 10 failed
[BOT-1] [OK] 40 requests sent | [FAIL] 12 failed
[BOT-1] [OK] 50 requests sent | [FAIL] 14 failed
[BOT-1] [OK] 60 requests sent | [FAIL] 16 failed
[BOT-1] [OK] 70 requests sent | [FAIL] 18 failed
[BOT-1] [FAIL] 20 requests failed (server overwhelmed!)
[BOT-1] [OK] 80 requests sent | [FAIL] 20 failed
[BOT-1] [OK] 90 requests sent | [FAIL] 22 failed
[BOT-1] [OK] 100 requests sent | [FAIL] 25 failed
[1]+  Running                  python3 ~/LINF02347/attacks/bot.py &
mininet>
```

Summary

#	Attack	Method	Protection - nftables
1	Network Scan	Scapy ICMP + TCP SYN	r2 - rate-limit ICMP, block non-public ports
2	FTP Brute Force	Python socket, wordlist	ftp -meter 3 connections/min/IP
3	ARP Cache Poisoning	Scapy fake ARP replies	ws2/ws3 - drop fake gateway ARP replies
4	Reflected DDoS	Scapy IP spoofing + UDP reflector	http - drop unsolicited UDP, counter proof
5	Botnet HTTP Flood	C&C + bots (ws2, ws3)	DMZ servers - rate-limit new conn/IP on service ports

Key Design Principles

All protections implemented with nftables • No rules block specific IP addresses • Normal network operation preserved after each protection